

MeanMaxTemp.docx

Exported: May 28, 2026

Summary

This technical guide provides the Java implementation for two **MapReduce programs** designed to process large-scale climate records from the National Climatic Data Center. The first application focuses on **temporal data aggregation** by isolating specific character offsets in raw text to compute the **average annual temperature**. A secondary, more complex algorithm demonstrates **statistical filtering** by identifying peak temperatures and calculating the **mean maximum value per month**. By utilizing dedicated **Mapper and Reducer classes**, the code illustrates how to handle data quality checks and perform **distributed mathematical computations** within the Hadoop ecosystem.

Keywords: MapReduce Programming, NCDC Weather Data, Average Temperature Calculation, Mean Max Temperature, Java Hadoop Development

Content

From the following link extract the weather data <https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all> Create a Map Reduce program to a) find average temperature for each year from NCDC data set

```
import org.apache.hadoop.fs.Path;import org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Job;import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

//Dataset: https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all
public class AverageDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Please Enter the input and output parameters");
            System.exit(-1);
        }
        Job job = new Job();
        job.setJarByClass(AverageDriver.class);
        job.setJobName("Max temperature");
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        job.setMapperClass(AverageMapper.class);
        job.setReducerClass(AverageReducer.class);
```

```

0 : 1);}}Mapper:package averagetemp_amit;import java.io.IOException;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.LongWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Mapper;public
class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{public static final int MISSING = 9999;public void map(LongWritable key, Text
value, Mapper<LongWritable, Text, Text, IntWritable>.Context context) throws
IOException, InterruptedException {int temperature;String line =
value.toString();String year = line.substring(15, 19);if (line.charAt(87) == '+')
{temperature = Integer.parseInt(line.substring(88, 92));} else {temperature =
Integer.parseInt(line.substring(87, 92));}String quality = line.substring(92, 93);if
(temperature != 9999 && quality.matches("[01459]"))context.write(new Text(year),
new IntWritable(temperature));}}Reducer:package averagetemp_amit;import
java.io.IOException;import org.apache.hadoop.io.IntWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Reducer;public
class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {public
void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable, Text,
IntWritable>.Context context) throws IOException, InterruptedException {int
max_temp = 0;int count = 0;for (IntWritable value : values) {max_temp +=
value.get();count++;}context.write(key, new IntWritable(max_temp / count));}}b)
find the mean max temperature for every month\MEAN MAX DRIVER package
meanmax; import org.apache.hadoop.fs.Path; import
org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.Text; import
org.apache.hadoop.mapreduce.Job; import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat; public class
MeanMaxDriver { public static void main(String[] args) throws Exception { if
(args.length != 2) { System.err.println("Please Enter the input and output
parameters"); System.exit(-1); } Job job = new Job();
job.setJarByClass(MeanMaxDriver.class); job.setJobName("Max temperature");
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.setMapperClass(MeanMaxMapper.class);
job.setReducerClass(MeanMaxReducer.class); job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
System.exit(job.waitForCompletion(true) ? 0 : 1); }}\MEAN MAX MAPPER package
meanmax; import java.io.IOException; import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable; import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper; public class MeanMaxMapper
extends Mapper { public static final int MISSING = 9999; public void
map(LongWritable key, Text value, Mapper.Context context) throws IOException,
InterruptedException { int temperature; String line = value.toString(); String month

```

```

= line.substring(19, 21); if (line.charAt(87) == '+') { temperature =
Integer.parseInt(line.substring(88, 92)); } else { temperature =
Integer.parseInt(line.substring(87, 92)); } String quality = line.substring(92, 93); if
(temperature != 9999 && quality.matches("[01459]")) context.write(new
Text(month), new IntWritable(temperature)); } } \MEAN MAX REDUCERpackage
meanmax;import                                     java.io.IOException;import
org.apache.hadoop.io.IntWritable;import             org.apache.hadoop.io.Text;import
org.apache.hadoop.mapreduce.Reducer;public class MeanMaxReducer extends
Reducer <Text,IntWritable,Text,IntWritable>{public void reduce(Text key,
Iterable<IntWritable> values, Reducer< Text,IntWritable,Text,IntWritable>.Context
context) throws IOException,InterruptedException {int max_temp = 0; int total_temp
= 0; int count = 0; int days = 0; for (IntWritable value : values) { int temp =
value.get(); if (temp > max_temp) max_temp = temp; count++; if (count == 3)
{ total_temp += max_temp; max_temp = 0; count = 0; days++; } }
context.write(key, new IntWritable(total_temp / days)); } }

```